

# Introduction to Git and GitHub



git



**Speakers:**

Rojan Manandhar

Suyog Dahal

# Table of contents.

- Problem formulation (A simple HTML page)
- Introduction to Git and GitHub
- Things you can do with Git/Github
- Creating a repository
- Add-commit-push
- Making changes locally
- Cloning a repository
- Making changes in the web app
- Pulling the changes to the local repository
- Show changes between 2 versions
- Restore to older versions (version control)

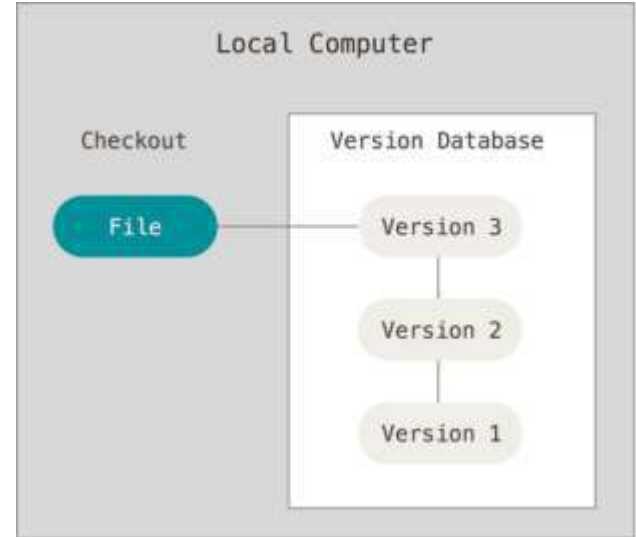
# What is Git?



- Created by Linus Torvalds, April 2005
- Software for tracking changes in a set of files
- Version Control System (VCS)
- Distributed version control
- Open source, free
- Used for coordinating work between multiple programmers

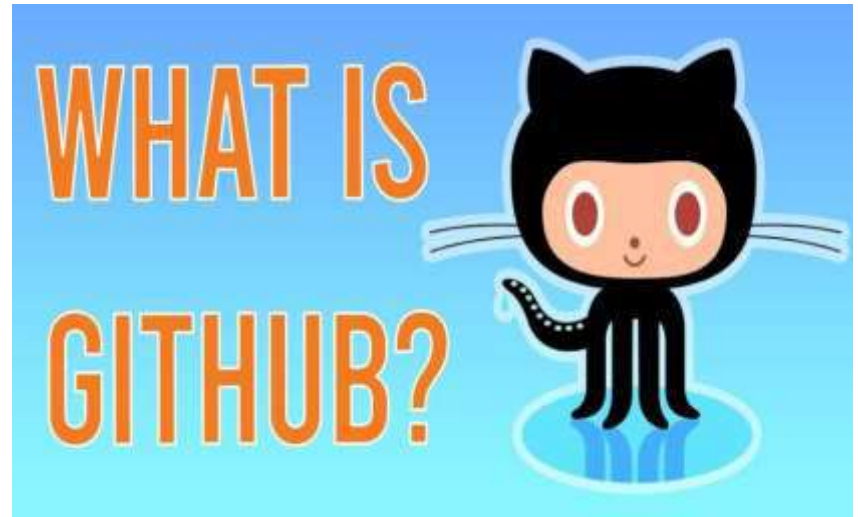
# What Is Version Control?

- Helps developers track and manage changes to a software project code.
- Lets developers safely work through branching and merging.
- Keeps track of every modification to the code.
- Lets you to revert your project back to an older version.



# What is GitHub?

- A web-based interface that uses [Git](#).
- Provides access control and several collaboration features
- Offers the distributed version control and source code management (SCM) functionality of Git
- If you have open-source projects that use Git, then GitHub is designed to help you better manage them.
- Code Hosting platform



# Benefits of using Git/Github

- It helps to keep the track of the code history
- GitHub is a place where project managers and developers come together to coordinate, track, and update their work so that projects are transparent and stay on schedule.
- All the code and documentation are in one place. There are millions of repositories on GitHub, and each repository has its own tools to help you host and release code.
- Collaborating is smoother and consumes lesser time.

# What is a repository?

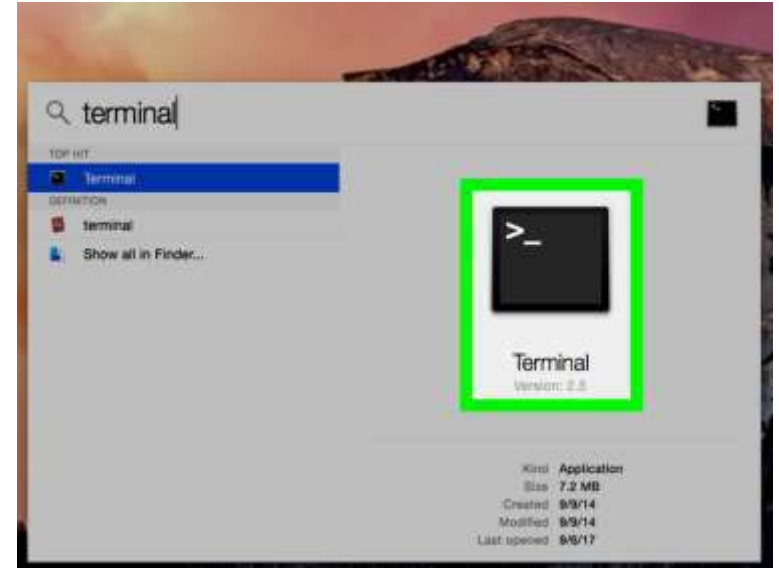
- A central location in which data is stored and managed.
- Usually used to organize a single project
- A repository contains all of your project's files and each file's revision history. You can discuss and manage your project's work within the repository.







The screenshot shows a Windows context menu for a folder. The menu items are: View, Sort by, Group by, Refresh, Customize this folder..., Paste, Paste shortcut, Git Bash Here (highlighted in blue), Graphics Properties..., Graphics Options, Share with, New, and Properties. A red arrow points to the 'Git Bash Here' option. The background shows a file explorer window with columns for Date modified and Type.



# Configuration

You typically configure your global username and email address after installing Git.

A terminal window titled "Terminal" with a dark background. The window shows two lines of text: the first line is the command "git config --global user.name 'suyogdahal'" and the second line is "git config --global user.email 'suyogdahal46@gmail.com'". The prompt "suyog@suyog-ubuntu ~/Desktop/demo \$" is visible before each command. The window has a standard Ubuntu-style title bar with a close button on the left and search and menu icons on the right.

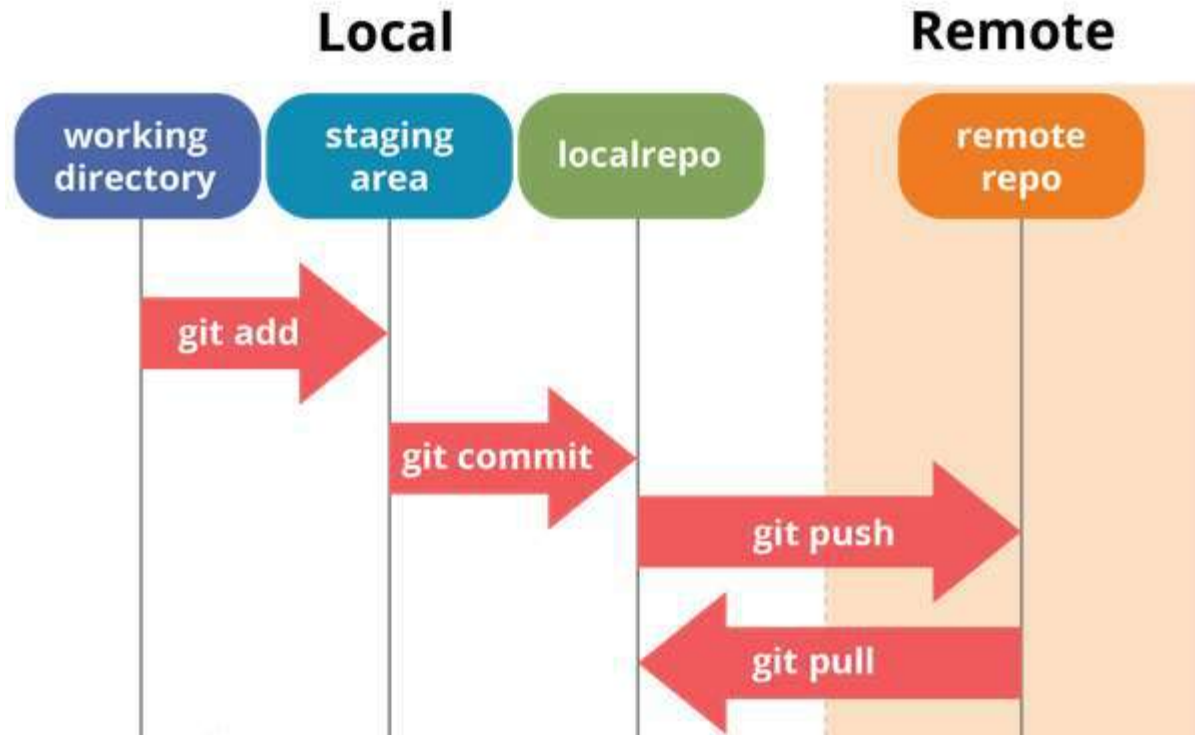
```
suyog@suyog-ubuntu ~/Desktop/demo $ git config --global user.name "suyogdahal"
suyog@suyog-ubuntu ~/Desktop/demo $ git config --global user.email "suyogdahal46@gmail.com"
```

# \$ git init

- The `git init` command is used to generate a new, empty Git repository or to reinitialize an existing one. It creates a new `.git` folder that contains all of the necessary Git metadata for the new repository.

```
suyog@suyog-ubuntu ~/Desktop/Demo $ git init  
Initialized empty Git repository in /home/suyog/Desktop/Demo/.git/
```

# Git Workflow



# Git follows following three tree architecture:

- **Working** – is the directory where the file changes are made.
- **Staging** – before the work has been committed to repository the changes are prepared for commit and sits in a staging index. Imagine a box. You can put stuff into the box. You can take stuff out of the box. This box is the staging area of Git. You can craft commits here. Committing is like sealing that box and sticking a label on it. The contents of that box are your changes.
- **Local Repository** – once the changes are staged these can be committed to repository that is in your local machine.

# \$ git status

- displays the state of the working directory and the staging area.

```
suyog@suyog-ubuntu ~/Desktop/Demo $ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    index.html
    vlc.jpg

nothing added to commit but untracked files present (use "git add" to track)
```

# States that file can be in

- **Untracked** means Git found a file you didn't have in previous commit.
- **Modified** means that we have changed the contents of file but still not committed on our local repo.
- **Staged** means that the modified is added to the staging area and is ready to go into next commit.
- **Committed** means that data is safely stored in local repo.

# \$ git add <filepath>

- This command tells Git to start tracking file by adding them to staging area.

```
suyog@suyog-ubuntu ~/Desktop/Demo $ git add index.html

suyog@suyog-ubuntu ~/Desktop/Demo $ git status
On branch main

No commits yet

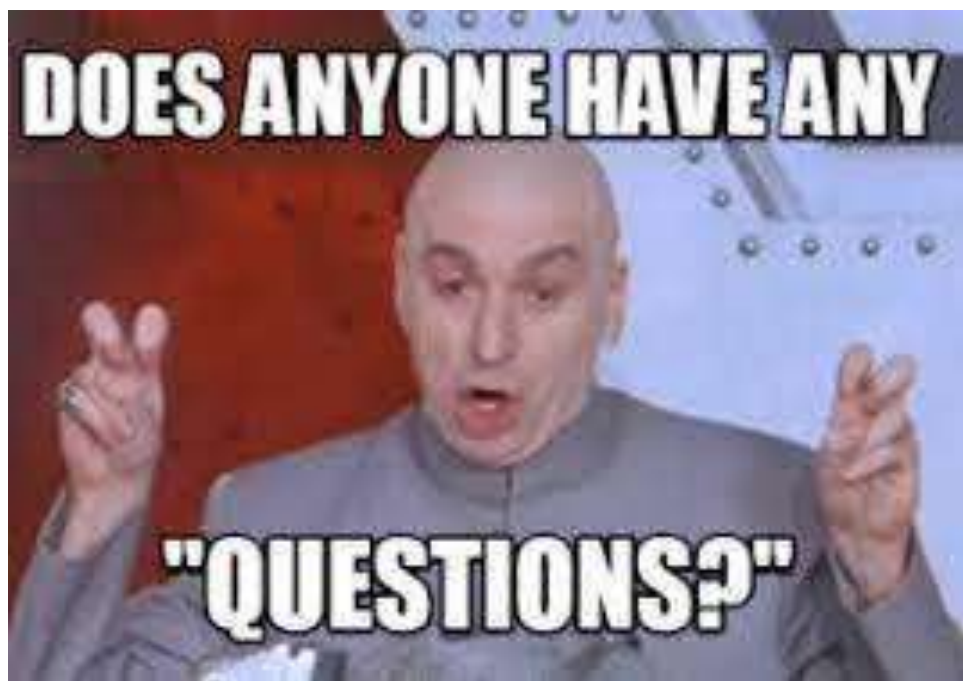
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        vlc.jpg
```

# \$ git commit -m “message”

- The git commit command captures a snapshot of the project's currently staged changes. Committed snapshots can be thought of as “safe” versions of a project—Git will never change them unless you explicitly ask it to..

```
suyog@suyog-ubuntu ~/Desktop/Demo $ git commit -m '1st feature added'
[main (root-commit) 1e06f16] 1st feature added
2 files changed, 20 insertions(+)
create mode 100644 index.html
create mode 100644 vlc.jpg
```



# Make changes locally

- Adding another feature feature 2



```
10 <h3> VLC Media Player </h3>
11
12 
13 <br><br>
14
15 <h2> i) 1st feature : ((PLAY)) </h2>
16 <h2> ii) 2nd feature : ((PAUSE)) </h2>
17
18 </center>
19 </Body>
20
21 </Html>
```

# Add and commit feature 2

```
suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git add .

suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git commit -m '2nd feature added'
[main 262e38f] 2nd feature added
1 file changed, 1 insertion(+)
```

# \$ git log

It shows a list of all the commits made to a repository. You can see the hash of each Git commit, the message associated with each commit, and more metadata. This command is useful for displaying the history of a repository.

```
suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git log
commit 262e38f7d4ca61be13ed78c4e34f6e2dc0a16b3d (HEAD -> main)
Author: suyogdahal <suyogdahal46@gmail.com>
Date:   Sun Jan 9 10:15:50 2022 +0545

    2nd feature added

commit 1e06f1607ea51aec9195a330e39604ba72fc24a4
Author: suyogdahal <suyogdahal46@gmail.com>
Date:   Sun Jan 9 10:10:10 2022 +0545

    1st feature added
```

# \$ git diff

- Displays the differences between 2 files.

```
suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git diff 1e06f16
diff --git a/index.html b/index.html
index c2e74f4..4b1a92a 100644
--- a/index.html
+++ b/index.html
@@ -13,6 +13,7 @@
         <br><br>

         <h2> i) 1st feature : ((PLAY)) </h2>
+       <h2> ii) 2nd feature : ((PAUSE)) </h2>^M
     </center>
 </Body>
```



# Restore to older version (version control)

\$ git checkout commit\_id -- file1/to/restore file2/to/restore

```
suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git log --oneline
262e38f (HEAD -> main) 2nd feature added
1e06f16 1st feature added

suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git checkout 1e06f16 .
Updated 1 path from 163ed50

suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html

suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git commit -m 'Restored to first version'
[main 7664804] Restored to first version
1 file changed, 1 deletion(-)
```



# Create a GitHub Repository

## Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#)

Owner <sup>\*</sup>

Repository name <sup>\*</sup>

 suyogdahal +

/ demo 

Great repository names are short and memorable. Need inspiration? How about [cuddly-telegram?](#)

**Description** (optional)

This repo is created for demo purpose for the Git/GitHub workshop conducted by IT and Robotics club of KEC

☒  **Public**

Anyone on the internet can see this repository. You choose who can control.

☐  **Private**

You choose who can see and commit to this repository.

**Initialize this repository with:**  
Skip this step if you're importing an existing repository.

☒ **Add a README file**

This is where you can write a long description for your project. [Learn more](#).

☒ **Add .gitignore**

Choose which files not to track from a list of templates. [Learn more](#).

☒ **Choose a license**

A license tells others what they can and can't do with your code. [Learn more](#).

Create repository

# Follow the steps to push local repo to remote

**Quick setup — if you've done this kind of thing before**

or [HTTPS](#) [SSH](#) `https://github.com/suyogdahal/demo.git`

Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.

**...or create a new repository on the command line**

```
echo "# demo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/suyogdahal/demo.git
git push -u origin main
```

**...or push an existing repository from the command line**

```
git remote add origin https://github.com/suyogdahal/demo.git
git branch -M main
git push -u origin main
```

**...or import code from another repository**

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

[Import code](#)

# \$ git push -u <remote\_repo> <branch>

- Git push is used for pushing contents of local repo to remote repo i.e GitHub

```
suyog@suyog-ubuntu ~/Desktop/Demo (main)$ git push --set-upstream origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 6.31 KiB | 6.31 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/suyogdahal/demo.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

# \$ git clone

- Cloning the remote repo to local.

```
suyog@suyog-ubuntu ~/Desktop $ git clone https://github.com/suyogdahal/demo.git
Cloning into 'demo'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 2), reused 8 (delta 2), pack-reused 0
Receiving objects: 100% (8/8), 6.31 KiB | 6.31 MiB/s, done.
Resolving deltas: 100% (2/2), done.
```

# Making changes in the github website



```
demo / index.html in main Cancel changes

<> Edit file Preview changes Tabs 8 No wrap

1 <html>
2 <head>
3 <title> GIT week1 </title>
4 </head>
5
6 <body>
7     <center>
8
9
10         <h3> VLC Media Player </h3>
11
12         
13         <br><br>
14
15         <h2> 1) 1st feature : {{PLAY}} </h2>
16         <h2> 11) 2nd feature : {{STOP}} </h2>
17
18     </center>
19 </body>
20
```

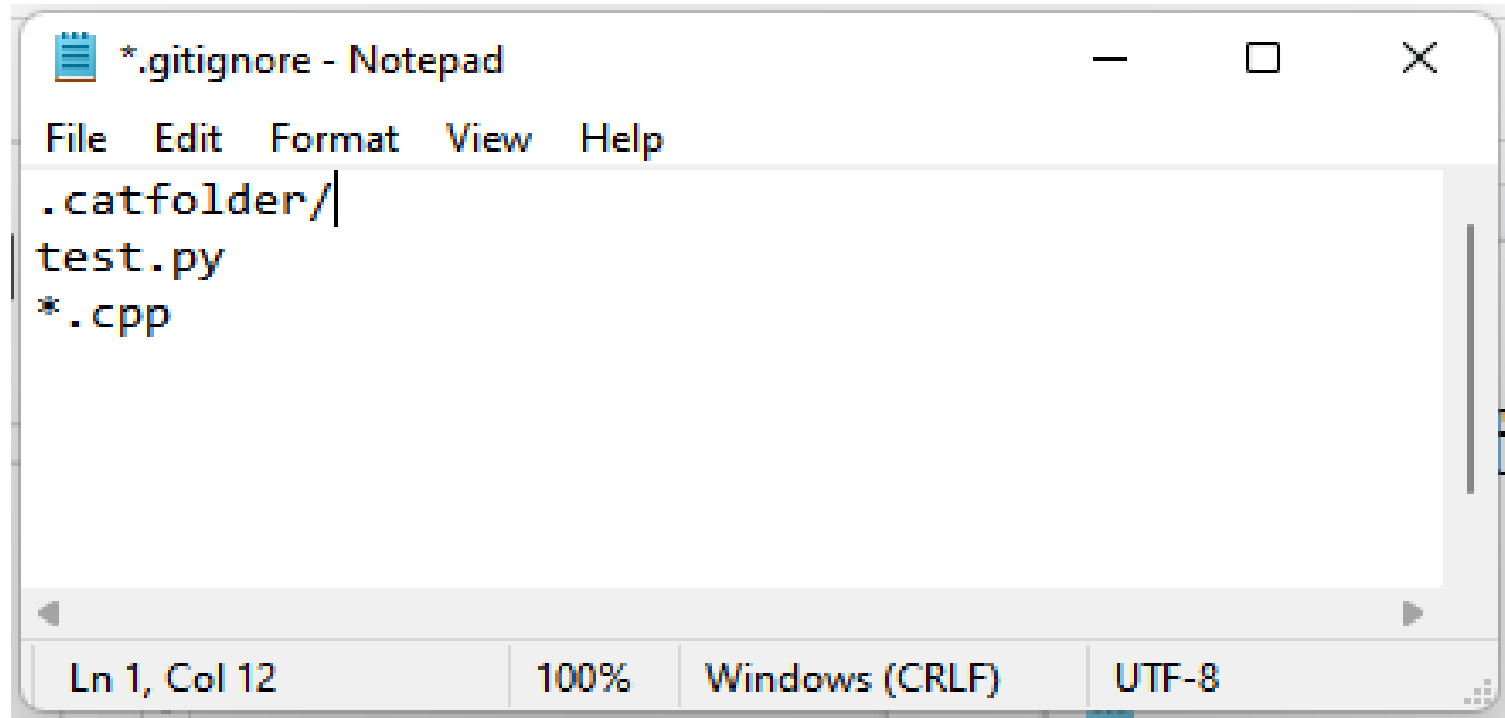
# \$ git pull

- Pull the changes from the remote repo (github) to the local repo.

```
suyog@suyog-ubuntu ~/Desktop/demo (main)$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 715 bytes | 715.00 KiB/s, done.
From https://github.com/suyogdahal/demo
   7664804..d17a077  main      -> origin/main
Updating 7664804..d17a077
Fast-forward
 index.html | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

# .gitignore file

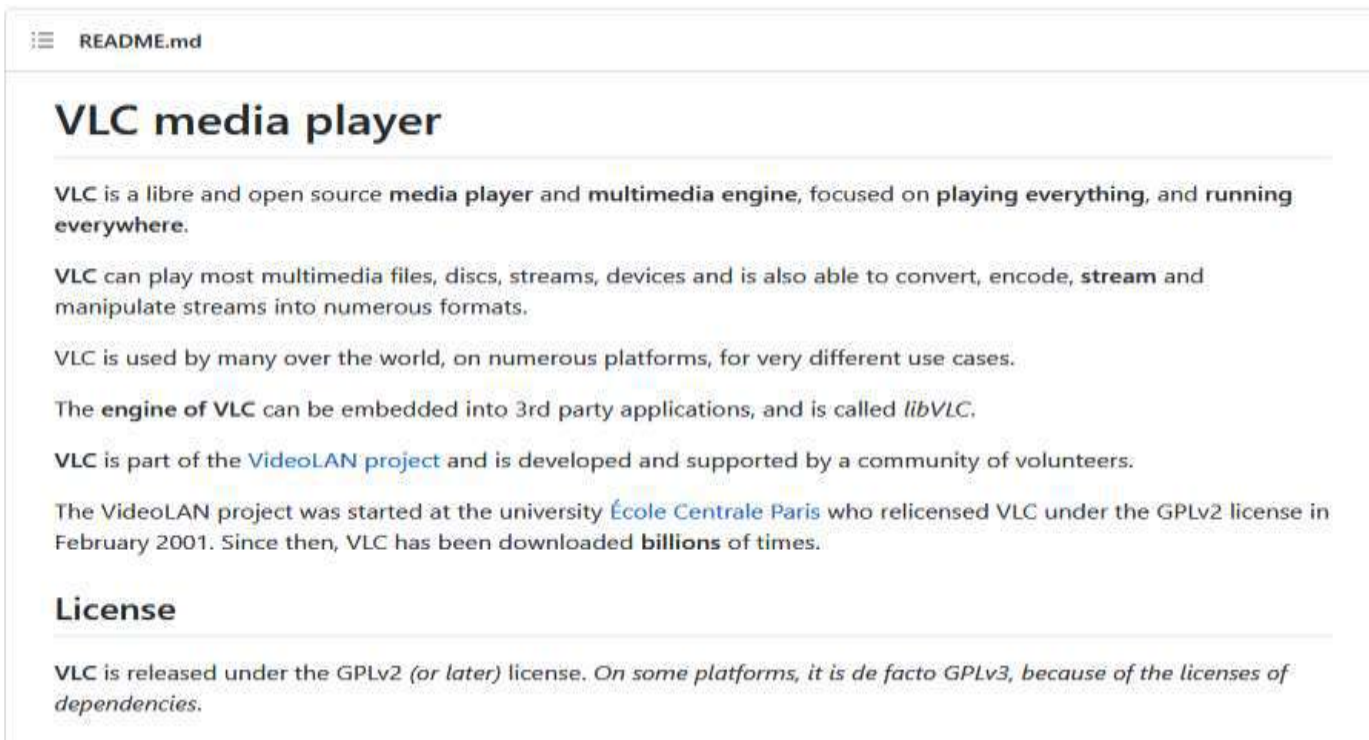
- The gitignore file is a file that tells git which files not to track (exclude) in the project

A screenshot of a Notepad window titled "\*.gitignore - Notepad". The window has a standard menu bar with "File", "Edit", "Format", "View", and "Help". The text area contains three lines of text: ".catfolder/", "test.py", and "\*.cpp". The status bar at the bottom shows "Ln 1, Col 12", "100%", "Windows (CRLF)", and "UTF-8".

```
*.gitignore - Notepad
File Edit Format View Help
.catfolder/
test.py
*.cpp
Ln 1, Col 12 100% Windows (CRLF) UTF-8
```

# README.md

- Document written in markdown language that describes the repository.



# Assignment

- For any one of your project written in any programming language, push it to a new github repository.

If you're facing any difficulties:

Contact us at:

**Facebook:** [Kec I.T. Club](#)

**Mail:** [kecktm.it.club@gmail.com](mailto:kecktm.it.club@gmail.com)

